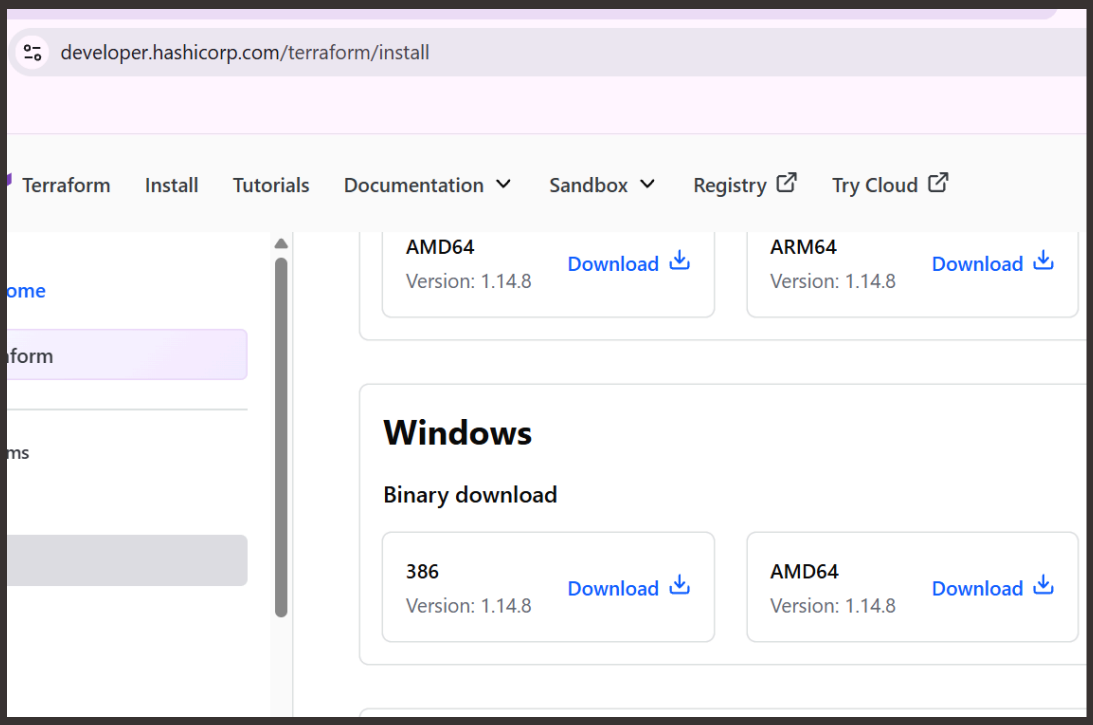
nextwork.org

Create S3 Buckets with Terraform

Danford Amos



Danford Amos
NextWork Student

Introducing Today's Project!

In this project, I will demonstrate the end-to-end process of using Terraform to automate the provisioning of an Amazon S3 bucket. The objective is to configure a fully functional Terraform workflow that includes initializing the working directory, validating the configuration, and executing an infrastructure plan and apply cycle. As part of the setup, we will diagnose and resolve any environment-level issues—such as installing and configuring the AWS CLI, verifying credentials, and ensuring Terraform can authenticate with AWS. Once the environment is properly configured, we will run terraform init, terraform plan, and terraform apply to generate an execution plan and deploy the S3 bucket according to the declared desired state.

Tools and concepts

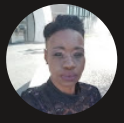
Amazon S3 — You learned how S3 stores objects, how bucket configuration works, how to upload/download files, and how to verify bucket metadata such as tags. AWS Identity and Access Management (IAM) — You used IAM credentials (via the AWS CLI) to authenticate Terraform and authorize resource creation. AWS CLI — You configured the CLI locally so Terraform could communicate with AWS using your credentials.

Project reflection

It took me a few hours to complete the project, including installing Terraform, configuring my AWS credentials, writing the initial S3 bucket configuration, applying the changes, and validating the uploaded object in the S3 console.

placeholder

I chose to do this project because I wanted hands-on experience with several core AWS and infrastructure-as-code tools: Terraform, Amazon S3, the AWS CLI, and AWS IAM.



Danford Amos
NextWork Student

nextwork.org

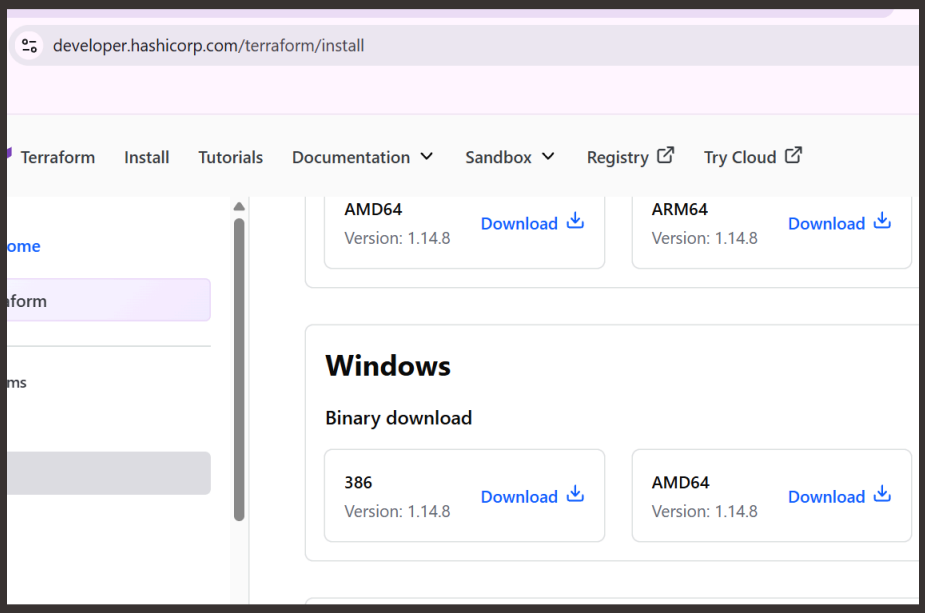
Working through the project gave me a practical way to understand how these services interact and how Terraform automates resource provisioning in AWS.

Introducing Terraform

You use Terraform to process a configuration file that defines the desired state of your infrastructure. Based on that configuration, Terraform determines which resources need to be created, updated, or deleted to bring the real infrastructure into alignment with the declared state. Terraform then executes those actions to build and maintain the desired environment.

Terraform is an Infrastructure-as-Code (IaC) tool, which means you manage your infrastructure using code instead of manually creating and configuring resources through the AWS Console or CLI. With IaC, Terraform automates the entire provisioning process by reading your configuration files and building the infrastructure exactly as defined.

The main.tf file serves as the primary entry point of a Terraform project and defines the desired state of your infrastructure. Within this file, you specify the resources, providers, and configurations that represent the target architecture you want Terraform to provision. When Terraform processes main.tf, it compares the declared desired state with the current state recorded in the state file and with the actual resources in your AWS environment. Based on this comparison, Terraform determines which resources already exist, which need to be created, and which require modification or deletion to align the real infrastructure with the configuration you defined.


developer.hashicorp.com/terraform/install
Terraform
Install
Tutorials
Documentation
Sandbox
Registry
Try Cloud
AMD64
Version: 1.14.8
Download
ARM64
Version: 1.14.8
Download
Windows
Binary download
386
Version: 1.14.8
Download
AMD64
Version: 1.14.8
Download

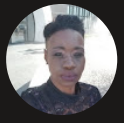
Danford Amos
NextWork Student

Configuration files

The configuration is structured in modulation. The advantage of doing this is the configuration is implemented using a modular architecture rather than a single monolithic block. By separating the infrastructure into three discrete modules, each containing its own logical set of resources, updates can be applied to an individual module without impacting the behavior or performance of the overall configuration. This structure also allows targeted modifications to main.tf, since the primary file simply orchestrates module calls instead of containing all resource definitions directly.

My main.tf configuration has three blocks

The configuration is structured in blocks instead of a single block, the advantage of doing this IE modularity is the ability to work on work on or update of the main.tf file, with the main part of There are 3 separate blocks of information called modulation, when your needing to change 1 part of your configuration with out affecting the performance of your configuration. The 1st block indicates that we are using AWS as our provider for this infrastructure. The 2nd block provisions an Amazon S3 bucket and gives it a unique name. The 3rd block manages the permissions or access of the S3 bucket Which means (blocking all public access).



Danford Amos
NextWork Student

nextwork.org

> Users > Dee > Desktop > network-terraform > main.tf

```tf
provider "aws" {
  region = "your-region-code"
}

resource "aws_s3_bucket" "my_bucket" {
  bucket = "nextwork-unique-bucket-danfordwilliams-1714" # Make sure this bucket name is globally unique by typing a lo
}

resource "aws_s3_bucket_public_access_block" "my_bucket_public_access_block" {
  bucket = aws_s3_bucket.my_bucket.id

  block_public_acls       = true
  ignore_public_acls      = true
  block_public_policy     = true
  restrict_public_buckets = true
}
```

Danford Amos
NextWork Student

Customizing my S3 Bucket

For my project extension, I reviewed the official Terraform documentation page to identify the configuration options available for customizing an S3 bucket. The documentation shows detailed examples, parameter references, and usage rules for each resource type. By examining these resource definitions, I was able to understand all configurable attributes, their valid values, and the constraints that govern how they can be used when defining the bucket in Terraform.

I customized my S3 bucket by adding tags so I can easily identify the project name and the purpose of the bucket in the future. After Terraform creates the bucket, I can verify that the tags were applied correctly by opening the S3 console, selecting the bucket, and reviewing the Tags section.

C: > Users > Dee > Desktop > network-terraform > main.tf

```
provider "aws" {
  region = "your-region-code"
}

resource "aws_s3_bucket" "my_bucket" {
  bucket = "nextwork-unique-bucket-danfordwilliams-1714" # Make sure this bucket name is globally unique by typi
}
tags = {
    Project = "Create an s3 Bucket with Terraform"
  }

resource "aws_s3_bucket_public_access_block" "my_bucket_public_access_block" {
  bucket = aws_s3_bucket.my_bucket.id

  block_public_acls       = true
  ignore_public_acls      = true
  block_public_policy     = true
  restrict_public_buckets = true
}
```

Danford Amos
NextWork Student

Terraform commands

In this step, we run the necessary Terraform commands init' to... initialize the working directory and generate an execution plan. Initializing Terraform downloads the required providers and prepares the project for use, while the planning phase analyzes the configuration file and shows the exact changes Terraform will make before any

Terraform plan gives you a preview of the exact changes Terraform intends to make, but with a bit more detail than the ultra-concise version: Terraform plan compares your configuration to the current state and shows the actions Terraform would take—such as creating, updating, or deleting resources—without actually applying them. It’s a safe way to verify that your configuration is correct before making any real changes.

```
              1 File(s)              0 bytes
              2 Dir(s)  423,505,780,736 bytes free

\Users\Dee\Desktop\network-terraform>terraform init
itializing the backend...
itializing provider plugins...

rraform has been successfully initialized!

u may now begin working with Terraform. Try running "terraform plan" t
see
y changes that are required for your infrastructure. All Terraform com
nds
ould now work.

 you ever set or change modules or backend configuration for Terraform

run this command to reinitialize your working directory. If you forget
other
mmands will detect it and remind you to do so if necessary.
```

Danford Amos
NextWork Student

AWS CLI and Access Keys

When we attempted to run terraform plan, Terraform returned a “no valid credentials found” error because our local terminal had not yet been configured with AWS credentials. Since the AWS provider plugin requires authenticated access to AWS APIs, Terraform cannot evaluate or interact with any AWS resources until valid credentials are available. To resolve this, we activated our AWS access keys and used the AWS CLI to configure them. By running aws configure and supplying the Access Key ID and Secret Access Key, we authenticated our local environment. Once these credentials were stored in the AWS CLI configuration, Terraform could successfully use them to authenticate with AWS and proceed with planning and applying changes.

The AWS CLI is a local tool that issues authenticated API calls to AWS, letting you manage cloud resources directly from your terminal instead of using the AWS Management Console.

I set up AWS access keys to ensure we have the credentials to connect to AWS, so now that our local computer has the credentials to aWS so does Terraform. When we ran Terraform plan it read our main.tf configuration file and it was able to successfully go to the AWS provider .

Danford Amos
NextWork Student


```
C:\Users\Dee\Desktop\network-terraform>aws --version
aws-cli/2.34.29 Python/3.14.3 Windows/11 exe/AMD64

C:\Users\Dee\Desktop\network-terraform>terraform plan

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your
configuration and found no differences, so no changes are needed.

C:\Users\Dee\Desktop\network-terraform>
```

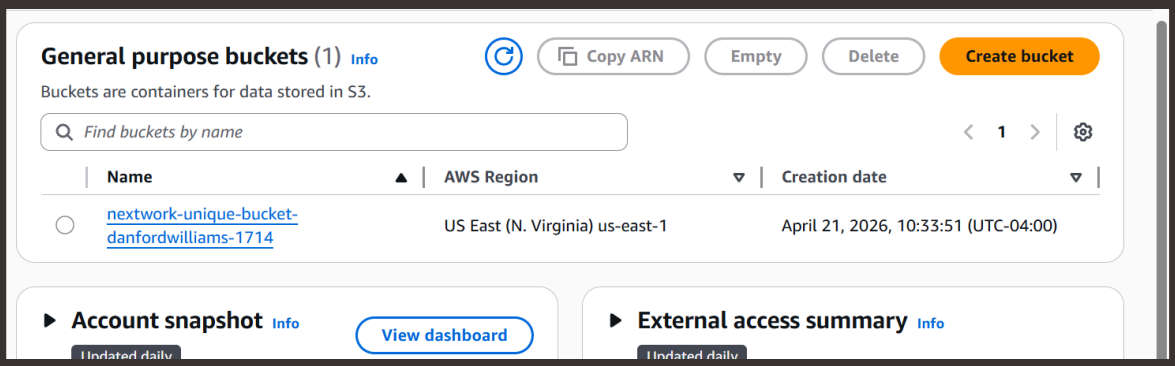
Lanching the S3 Bucket

We ran terraform apply to execute the changes that Terraform previously outlined during the planning phase. The apply step takes the actions shown in the plan and issues the corresponding AWS API calls to create or modify infrastructure. In this case, applying the configuration affects your AWS account by provisioning two new resources: an S3 bucket and its associated public-access-block configuration, which enforces the bucket’s permission settings.

The sequence of running terraform init, plan, and apply is crucial because, the sequence matters because each command enables the next: Init sets up the working directory and installs providers. Plan analyzes the configuration and shows the exact actions Terraform will take. Apply executes those actions against your AWS account. Running them in order ensures Terraform is initialized, the changes are known, and only then applied.

Danford Amos
NextWork Student

nextwork.org



Uploading an S3 Object

I added the aws_s3_object resource block so Terraform can manage the uploaded file as part of the infrastructure state. Buckets and objects are separate AWS resources, so defining the object explicitly lets Terraform track its checksum, detect drift, and ensure the file is created or updated consistently during each apply.

We need to run Terraform apply again because we updated our Terraform configuration. Any change to the .tf files requires Terraform to re-evaluate the desired state and compare it with the current infrastructure recorded in the state file. After running a new plan, Terraform determined that only one additional resource needs to be created based on the updated configuration, and apply is the step that executes that change..

To validate that our updated configuration worked correctly, we inspected the S3 bucket and confirmed that the newly uploaded image was present. We then downloaded the object back to our local machine to verify that it matched the original file, ensuring that both the upload process and the bucket configuration behaved as expected.

Danford Amos
NextWork Student

nextwork.org

C: > Users > Dee > Desktop > network-terraform > main.tf

```
resource "aws_s3_bucket_public_access_block" "my_bucket_public_access_block" {
  bucket = aws_s3_bucket.my_bucket.id

  block_public_acls       = true
  ignore_public_acls      = true
  block_public_policy     = true
  restrict_public_buckets = true
}
resource "aws_s3_object" "image" {
  bucket = aws_s3_bucket.my_bucket.id # Reference the bucket ID
  key    = "image.png" # Path in the bucket
  source = "image.png" # Local file path
}
```

nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects